

MySQL / MariaDB

Performance Tuning Runbook

Query Tuning | Parameter Tuning | Database Tuning

Extended Edition

Document Type	Operational Runbook — Performance (Extended)
Scope	MySQL 5.7+ / 8.x & MariaDB 10.x+
Audience	DBA / SRE / Infrastructure Engineers
Classification	Internal — Restricted
Filesystem Layout	
Data Directory	/apps/mysql/data
Binary Logs	/apps/mysql/binlog
Backups	/apps/mysql/backups
Error Log	/var/log/mysqld.log

Table of Contents

- **Pre-Scenario: Triage & Classification**
- **Scenario 1: Slow Query Identification & Index Optimization**
 - 1.1 Finding the Issue
 - 1.2 Troubleshooting / Remediation
 - 1.3 Confirmation
- **Scenario 2: InnoDB Buffer Pool & Memory Parameter Tuning**
 - 2.1 Finding the Issue
 - 2.2 Troubleshooting / Remediation
 - 2.3 Confirmation
- **Scenario 3: Table & Index Bloat — Database-Level Tuning**
 - 3.1 Finding the Issue
 - 3.2 Troubleshooting / Remediation
 - 3.3 Confirmation
- **Scenario 4: Connection Storms & Thread Exhaustion**
 - 4.1 Finding the Issue
 - 4.2 Troubleshooting / Remediation
 - 4.3 Confirmation
- **Scenario 5: Disk I/O Bottleneck & Redo Log / Flush Tuning**
 - 5.1 Finding the Issue
 - 5.2 Troubleshooting / Remediation
 - 5.3 Confirmation
- **Scenario 6: Lock Contention & Deadlock Resolution**
 - 6.1 Finding the Issue
 - 6.2 Troubleshooting / Remediation
 - 6.3 Confirmation
- **Scenario 7: Replication Lag Diagnosis & Tuning**
 - 7.1 Finding the Issue
 - 7.2 Troubleshooting / Remediation

7.3 Confirmation

- **Scenario 8: Slow DDL / Schema Changes on Large Tables**

- 8.1 Finding the Issue

- 8.2 Troubleshooting / Remediation

- 8.3 Confirmation

- **Scenario 9: Binary Log Growth & Disk Space Exhaustion**

- 9.1 Finding the Issue

- 9.2 Troubleshooting / Remediation

- 9.3 Confirmation

- **Scenario 10: Security Misconfig, Privilege Bloat & Audit Overhead**

- 10.1 Finding the Issue

- 10.2 Troubleshooting / Remediation

- 10.3 Confirmation

Pre-Scenario: How to Classify the Performance Issue Type

Run this triage to classify the root cause before diving into scenarios.

Step A: Single Offending Query?

```
$ tail -200 /apps/mysql/data/slow-query.log

mysql> SELECT DIGEST_TEXT, COUNT_STAR, SUM_TIMER_WAIT/1000000000 AS total_ms
FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC LIMIT 10;
```

One query dominates? = **Scenario 1**.

Step B: Server Parameters Inadequate?

```
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
mysql> SHOW GLOBAL STATUS LIKE 'Created_tmp%';
mysql> SHOW GLOBAL STATUS LIKE 'Opened_tables';
```

Low hit rate / disk spills? = **Scenario 2** or **4**.

Step C: System Resources Exhausted?

```
$ iostat -xm 1 5
$ free -h
$ vmstat 1 5
$ df -h /apps/mysql/data /apps/mysql/binlog
```

I/O saturated? = **5**. Disk full? = **9**. Lock waits? = **6**. Replication lag? = **7**. DDL stuck? = **8**.

Scenario 1: Slow Query Identification & Index Optimization

The most common performance problem. A poorly optimized query does full table scans or uses sub-optimal execution plans.

① Phase 1 — How to Find the Issue

Step 1: Enable and review the Slow Query Log:

```
mysql> SHOW VARIABLES LIKE 'slow_query%';
mysql> SET GLOBAL slow_query_log = 'ON';
mysql> SET GLOBAL slow_query_log_file = '/apps/mysql/data/slow-query.log';
mysql> SET GLOBAL long_query_time = 1;
mysql> SET GLOBAL log_queries_not_using_indexes = 'ON';
```

Step 2: Aggregate slow query patterns with mysqldumpslow:

```
$ mysqldumpslow -s t -t 10 /apps/mysql/data/slow-query.log
$ mysqldumpslow -s at -t 10 /apps/mysql/data/slow-query.log
$ mysqldumpslow -s c -t 10 /apps/mysql/data/slow-query.log
```

Step 3: Deep analysis with pt-query-digest:

```
$ pt-query-digest /apps/mysql/data/slow-query.log > /tmp/query_report.txt
$ pt-query-digest --since "1h" /apps/mysql/data/slow-query.log
```

Step 4: EXPLAIN / EXPLAIN ANALYZE the identified query:

```
mysql> EXPLAIN SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.order_date > '2026-01-01' AND c.region = 'WEST';

mysql> EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'pending' AND total > 500;
```

Red Flags: type=ALL, key=NULL, Extra="Using filesort"/"Using temporary", high rows value.

② Phase 2 — Troubleshooting / Remediation

Step 1: Add missing indexes:

```
mysql> ALTER TABLE orders ADD INDEX idx_status_date (status, order_date, total);

# Covering index:
mysql> ALTER TABLE orders ADD INDEX idx_cov (customer_id, order_date, status, total);

# Online DDL:
mysql> ALTER TABLE orders ADD INDEX idx_region (region), ALGORITHM=INPLACE,
LOCK=NONE;
```

Step 2: Rewrite query patterns:

```
# BAD: Function on indexed column
mysql> SELECT * FROM orders WHERE YEAR(order_date) = 2026;

# GOOD: Range scan
mysql> SELECT * FROM orders WHERE order_date >= '2026-01-01' AND order_date <
'2027-01-01';

# BAD: SELECT *
mysql> SELECT * FROM orders WHERE status = 'pending';

# GOOD: Select needed columns only
mysql> SELECT order_id, order_date, total FROM orders WHERE status = 'pending';
```

Step 3: Use query hints / update stats:

```
mysql> SELECT * FROM orders FORCE INDEX (idx_status_date)
WHERE status = 'shipped' AND order_date > '2026-01-01';

mysql> ANALYZE TABLE orders;
mysql> SET GLOBAL innodb_stats_persistent_sample_pages = 40;
```

③ Phase 3 — Confirmation

Step 1: Re-run EXPLAIN — verify index usage:

```
mysql> EXPLAIN SELECT order_id, order_date, total FROM orders
WHERE status = 'pending' AND order_date > '2026-01-01';
# type=ref/range, key=idx_status_date
```

Step 2: Check slow log for absence of the query:

```
$ tail -50 /apps/mysql/data/slow-query.log | grep "orders"

mysql> SHOW GLOBAL STATUS LIKE 'Slow_queries';
```

CONFIRMED: EXPLAIN shows index usage, query time dropped, no new slow queries.

Scenario 2: InnoDB Buffer Pool & Memory Parameter Tuning

When queries are well-indexed but still slow, the server may be memory-starved. The buffer pool is the single most impactful parameter.

① Phase 1 — How to Find the Issue

Step 1: Calculate buffer pool hit rate:

```
mysql> SELECT (1 - (SELECT VARIABLE_VALUE FROM performance_schema.global_status
WHERE VARIABLE_NAME = 'Innodb_buffer_pool_reads') /
(SELECT VARIABLE_VALUE FROM performance_schema.global_status
WHERE VARIABLE_NAME = 'Innodb_buffer_pool_read_requests'))
* 100 AS hit_pct;
# Target: > 99%
```

Step 2: Check temp table spills and swap:

```
mysql> SHOW GLOBAL STATUS LIKE 'Created_tmp%';
# Disk ratio target: < 10%

$ free -h
$ cat /proc/$(pidof mysqld)/status | grep VmSwap
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Resize buffer pool (online):

```
# Rule: 70-80% of RAM
mysql> SET GLOBAL innodb_buffer_pool_size = 25769803776; -- 24GB
mysql> SHOW STATUS LIKE 'Innodb_buffer_pool_resize_status';

# /etc/my.cnf: innodb_buffer_pool_size = 24G
# innodb_buffer_pool_instances = 8
```

Step 2: Tune temp/sort buffers:

```
mysql> SET GLOBAL tmp_table_size = 268435456; -- 256MB
mysql> SET GLOBAL max_heap_table_size = 268435456;
mysql> SET GLOBAL sort_buffer_size = 4194304; -- 4MB
```

WARNING: Per-connection buffers multiply by max_connections! 500 x 4MB = 2GB.

Step 3: Reduce swappiness:

```
$ sysctl vm.swappiness=1  
$ echo "vm.swappiness = 1" >> /etc/sysctl.conf
```

③ Phase 3 — Confirmation

Step 1: Re-check hit rate (after 15-30 min):

```
mysql> FLUSH STATUS;  
# Wait, then re-query hit_pct – should be > 99%
```

Step 2: Verify zero swap:

```
$ vmstat 1 5 # si and so = 0  
$ cat /proc/$(pidof mysqld)/status | grep VmSwap
```

CONFIRMED: Buffer pool hit > 99%, temp disk < 10%, zero swap.

Scenario 3: Table & Index Bloat — Database-Level Tuning

InnoDB tables accumulate fragmentation from DELETES and UPDATES, wasting buffer pool memory and slowing scans.

① Phase 1 — How to Find the Issue

Step 1: Identify fragmented tables:

```
mysql> SELECT TABLE_SCHEMA, TABLE_NAME,  
ROUND(DATA_LENGTH/1024/1024,2) AS data_mb, ROUND(DATA_FREE/1024/1024,2) AS free_mb,  
ROUND(DATA_FREE/(DATA_LENGTH+1)*100,1) AS frag_pct  
FROM information_schema.TABLES  
WHERE TABLE_SCHEMA NOT IN ('mysql','sys','performance_schema','information_schema')  
AND DATA_FREE > 10*1024*1024 ORDER BY DATA_FREE DESC;
```

Step 2: Check for duplicate/unused indexes:

```
$ pt-duplicate-key-checker --host=localhost --user=root --ask-pass  
  
mysql> SELECT * FROM sys.schema_unused_indexes WHERE object_schema = 'your_database';
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Defragment tables:

```
mysql> OPTIMIZE TABLE your_database.bloated_table;  
# Or: ALTER TABLE your_database.bloated_table ENGINE=InnoDB;
```

WARNING: OPTIMIZE TABLE acquires metadata lock. Schedule during maintenance windows.

Step 2: Remove unused indexes and archive old data:

```
mysql> ALTER TABLE orders DROP INDEX idx_old_1, DROP INDEX idx_old_2;  
  
mysql> CREATE TABLE orders_archive LIKE orders;  
mysql> INSERT INTO orders_archive SELECT * FROM orders WHERE order_date <  
'2024-01-01' LIMIT 10000;  
mysql> DELETE FROM orders WHERE order_date < '2024-01-01' LIMIT 10000;  
# Repeat, then: ALTER TABLE orders ENGINE=InnoDB;
```

③ Phase 3 — Confirmation

Step 1: Re-check fragmentation — should be < 5%:

```
mysql> SELECT TABLE_NAME, ROUND(DATA_FREE/(DATA_LENGTH+1)*100,1) AS frag_pct
FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'your_database' ORDER BY
DATA_FREE DESC;
```

Step 2: Verify file sizes on disk:

```
$ du -sh /apps/mysql/data/your_database/
```

CONFIRMED: Fragmentation < 5%, .ibd files reduced, backup size decreased.

Scenario 4: Connection Storms & Thread Exhaustion

Too many connections or connection leaks cause 'Too many connections' errors and cascading application failures.

① Phase 1 — How to Find the Issue

Step 1: Check current vs max connections:

```
mysql> SHOW VARIABLES LIKE 'max_connections';
mysql> SHOW GLOBAL STATUS LIKE 'Max_used_connections';
mysql> SHOW GLOBAL STATUS LIKE 'Threads_connected';
mysql> SHOW GLOBAL STATUS LIKE 'Aborted_connects';
```

Step 2: Find sleeping/leaked connections:

```
mysql> SELECT USER, SUBSTRING_INDEX(HOST, ':', 1) AS ip, COUNT(*) AS cnt
FROM information_schema.PROCESSLIST GROUP BY USER, ip ORDER BY cnt DESC;

mysql> SELECT ID, USER, HOST, TIME FROM information_schema.PROCESSLIST
WHERE COMMAND = 'Sleep' AND TIME > 300 ORDER BY TIME DESC;
```

Step 3: Check error log:

```
$ grep -i "too many connections" /var/log/mysqld.log
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Kill idle connections:

```
$ mysql -u root -p -N -e "SELECT CONCAT('KILL ', ID, ';') FROM
information_schema.PROCESSLIST
WHERE COMMAND = 'Sleep' AND TIME > 600" | mysql -u root -p
```

Step 2: Tune parameters:

```
mysql> SET GLOBAL max_connections = 500;
mysql> SET GLOBAL wait_timeout = 300;
mysql> SET GLOBAL interactive_timeout = 600;
mysql> SET GLOBAL thread_cache_size = 64;
```

Step 3: Set per-user limits:

```
mysql> ALTER USER 'app_user'@'%' WITH MAX_USER_CONNECTIONS 100;
```

③ Phase 3 — Confirmation

Step 1: Verify stabilization:

```
mysql> SHOW GLOBAL STATUS LIKE 'Threads_connected';  
mysql> SHOW GLOBAL STATUS LIKE 'Aborted_connects';  
# Aborted_connects should stop increasing
```

Step 2: Confirm error log clean:

```
$ tail -f /var/log/mysqld.log | grep -i "too many"
```

CONFIRMED: No "Too many connections", threads stable, idle connections auto-cleaned.

Scenario 5: Disk I/O Bottleneck & Redo Log / Flush Tuning

High write workloads saturate disk I/O. InnoDB redo log flushes, doublewrite, and checkpoints compete for bandwidth.

① Phase 1 — How to Find the Issue

Step 1: Check OS disk I/O:

```
$ iostat -xm 1 10
# %util > 80% = saturated, await > 20ms = queuing
$ mpstat 1 5 # %iowait > 10% = CPU waiting on disk
```

Step 2: Check redo log pressure:

```
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_log_waits';
# > 0 = redo log too small

mysql> SHOW VARIABLES LIKE 'innodb_log_file_size';
mysql> SHOW VARIABLES LIKE 'innodb_flush_log_at_trx_commit';
mysql> SHOW VARIABLES LIKE 'innodb_flush_method';
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Tune innodb_io_capacity:

```
# HDD=200, SAS=400, SSD=2000, NVMe=10000-20000
mysql> SET GLOBAL innodb_io_capacity = 2000;
mysql> SET GLOBAL innodb_io_capacity_max = 4000;
```

Step 2: Increase redo log size:

```
# MySQL 5.7 (restart): innodb_log_file_size = 1G
$ systemctl stop mysql
$ mv /apps/mysql/data/ib_logfile0 /apps/mysql/backups/ib_logfile0.bak
$ mv /apps/mysql/data/ib_logfile1 /apps/mysql/backups/ib_logfile1.bak
$ systemctl start mysql

# MySQL 8.0.30+ (online):
mysql> SET GLOBAL innodb_redo_log_capacity = 2147483648; -- 2GB
```

Step 3: Optimize flush method:

```
mysql> SET GLOBAL innodb_flush_method = 'O_DIRECT';  
mysql> SET GLOBAL innodb_flush_log_at_trx_commit = 2; -- fast, 1s risk
```

WARNING: innodb_flush_log_at_trx_commit=2 can lose 1 second of data in a crash. Only use on replicated setups.

③ Phase 3 — Confirmation

Step 1: Verify I/O dropped:

```
$ iostat -xm 1 10 # %util < 60%, await < 10ms
```

Step 2: Verify log waits zero:

```
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_log_waits';  
# 0 or not increasing  
$ ls -lh /apps/mysql/data/ib_logfile*
```

CONFIRMED: Disk %util < 60%, Innodb_log_waits=0, write TPS improved.

Scenario 6: Lock Contention & Deadlock Resolution

Concurrent transactions competing for the same rows cause lock waits and deadlocks. Symptoms include rising latency, threads stuck in 'LOCK WAIT' state, and application timeout errors.

① Phase 1 — How to Find the Issue

Step 1: Check current lock waits (InnoDB):

```
mysql> SHOW ENGINE INNODB STATUS\G
# Focus on: LATEST DETECTED DEADLOCK and TRANSACTIONS

# Direct lock wait query:
mysql> SELECT r.trx_id AS waiting_trx, r.trx_mysql_thread_id AS waiting_pid,
r.trx_query AS waiting_query, b.trx_mysql_thread_id AS blocking_pid,
b.trx_query AS blocking_query
FROM information_schema.INNODB_LOCK_WAITS w
JOIN information_schema.INNODB_TRX b ON b.trx_id = w.blocking_trx_id
JOIN information_schema.INNODB_TRX r ON r.trx_id = w.requesting_trx_id;
```

MySQL 8.0+: Use performance_schema.data_lock_waits and data_locks instead.

Step 2: Check metadata lock waiters:

```
mysql> SELECT * FROM performance_schema.metadata_locks WHERE LOCK_STATUS = 'PENDING';

mysql> SELECT ID, USER, HOST, TIME, STATE, INFO FROM information_schema.PROCESSLIST
WHERE STATE LIKE '%lock%';
```

Step 3: Check deadlock frequency:

```
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_row_lock%';

$ grep -A 50 'LATEST DETECTED DEADLOCK' /var/log/mysqld.log | head -60
```

Step 4: Use sys schema for blocking chain:

```
mysql> SELECT * FROM sys.innodb_lock_waits\G
# Shows: waiting_query, blocking_query, blocking_pid, wait_age
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Kill blocking transaction:

```
mysql> KILL 12345; -- blocking_pid from previous step

mysql> SELECT CONCAT('KILL ', trx_mysql_thread_id, ';') FROM
information_schema.INNODB_TRX
WHERE trx_started < NOW() - INTERVAL 5 MINUTE;
```

WARNING: KILL causes ROLLBACK which can take time on large DML. Monitor Threads_running after.

Step 2: Enable deadlock logging:

```
mysql> SET GLOBAL innodb_print_all_deadlocks = ON;
# Every deadlock now logged to /var/log/mysqld.log
```

Step 3: Fix root cause — reorder operations & shorten transactions:

```
# Rule 1: Always access tables in SAME ORDER across all transactions
# Rule 2: Keep transactions SHORT – move non-DB work outside
# Rule 3: Use proper indexes on WHERE in UPDATE/DELETE

mysql> EXPLAIN UPDATE orders SET status = 'shipped'
WHERE customer_id = 42 AND order_date > '2026-01-01';
# type should NOT be ALL – add index if missing:
mysql> ALTER TABLE orders ADD INDEX idx_cust_date (customer_id, order_date);

# Rule 4: Consider READ COMMITTED isolation:
mysql> SET GLOBAL transaction_isolation = 'READ-COMMITTED';
```

Step 4: Tune lock wait timeout:

```
mysql> SET GLOBAL innodb_lock_wait_timeout = 10;
# App must handle Error 1205 and retry
```

③ Phase 3 — Confirmation

Step 1: Monitor lock waits (should decline):

```
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_row_lock_waits';
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_row_lock_time_avg';
```

Step 2: Verify no new deadlocks:

```
$ tail -f /var/log/mysqld.log | grep -i "deadlock"
# No new entries for 30+ min
```

Step 3: No threads stuck in lock wait:


```
mysql> SELECT COUNT(*) FROM information_schema.PROCESSLIST WHERE STATE LIKE '%lock%';  
# Should be 0
```

CONFIRMED: Lock waits near zero, no new deadlocks, lock_time_avg dropped, app errors resolved.

Scenario 7: Replication Lag Diagnosis & Tuning

Replication lag occurs when the replica cannot apply relay logs as fast as the source produces them. This causes stale reads, reporting delays, and failover risk.

① Phase 1 — How to Find the Issue

Step 1: Check replication status and lag:

```
# On REPLICA:
mysql> SHOW SLAVE STATUS\G
# Key: Slave_IO_Running=Yes, Slave_SQL_Running=Yes
# Seconds_Behind_Master should be 0

# MySQL 8.0.22+: SHOW REPLICA STATUS\G
```

Step 2: Identify IO vs SQL thread bottleneck:

```
# IO_Running=Yes but lag high = SQL thread (apply) bottleneck
# IO_Running=Connecting = network/source issue

$ ls -lt /apps/mysql/data/relay-log.* | head -10
$ mysqladmin -u root -p extended-status -i 5 | grep "Seconds_Behind"
```

Step 3: Check if single-threaded apply is the bottleneck:

```
mysql> SHOW VARIABLES LIKE 'slave_parallel_workers';
# 0 = single-threaded (default) = SLOW
mysql> SHOW VARIABLES LIKE 'slave_parallel_type';
```

Common lag causes: Large DELETE without LIMIT, DDL on big tables, single-threaded apply, reporting queries blocking DDL replay on replica.

② Phase 2 — Troubleshooting / Remediation

Step 1: Enable multi-threaded replication:

```
mysql> STOP SLAVE;
mysql> SET GLOBAL slave_parallel_workers = 8;
mysql> SET GLOBAL slave_parallel_type = 'LOGICAL_CLOCK';
mysql> START SLAVE;

# MySQL 8.0.27+ (WRITESET):
mysql> STOP REPLICA;
mysql> SET GLOBAL replica_parallel_workers = 8;
mysql> SET GLOBAL binlog_transaction_dependency_tracking = 'WRITESET';
mysql> START REPLICA;
```

Step 2: Optimize large transactions on source:

```
# BAD: Giant delete
mysql> DELETE FROM logs WHERE created_at < '2025-01-01';

# GOOD: Batch deletes
mysql> DELETE FROM logs WHERE created_at < '2025-01-01' LIMIT 10000;
# Repeat with SLEEP(0.5) between batches

mysql> SET GLOBAL binlog_row_image = 'MINIMAL';
# Reduces binlog size by 50-80%
```

Step 3: Skip errors (emergency only):

```
mysql> STOP SLAVE;
mysql> SET GLOBAL sql_slave_skip_counter = 1;
mysql> START SLAVE;

# GTID-based: inject empty transaction
mysql> STOP SLAVE;
mysql> SET GTID_NEXT = 'source-uuid:problematic-gtid';
mysql> BEGIN; COMMIT;
mysql> SET GTID_NEXT = 'AUTOMATIC';
mysql> START SLAVE;
```

WARNING: Skipping events causes data drift. Verify consistency with `pt-table-checksum` afterwards.

③ Phase 3 — Confirmation

Step 1: Verify lag drops to 0:

```
mysql> SHOW SLAVE STATUS\G
# Seconds_Behind_Master: 0

$ watch -n 5 "mysql -u root -p -e 'SHOW SLAVE STATUS\G' | grep Seconds"
```

Step 2: Verify data consistency:

```
$ pt-table-checksum --replicate=percona.checksums --databases=your_database  
--host=localhost -u root --ask-pass
```

CONFIRMED: Seconds_Behind_Master=0, parallel workers active, pt-table-checksum shows no drift.

Scenario 8: Slow DDL / Schema Changes on Large Tables

ALTER TABLE on multi-billion-row tables can take hours, blocking writes. Some operations still require full table copies.

① Phase 1 — How to Find the Issue

Step 1: Check for ongoing ALTER operations:

```
mysql> SELECT * FROM information_schema.PROCESSLIST
WHERE STATE LIKE '%alter%' OR STATE LIKE '%copy%' OR INFO LIKE '%ALTER%';

# Progress (MySQL 8.0+):
mysql> SELECT EVENT_NAME, WORK_COMPLETED, WORK_ESTIMATED,
ROUND(WORK_COMPLETED/WORK_ESTIMATED*100, 2) AS pct_done
FROM performance_schema.events_stages_current WHERE EVENT_NAME LIKE '%alter%';
```

Step 2: Estimate ALTER duration:

```
mysql> SELECT TABLE_NAME, TABLE_ROWS,
ROUND((DATA_LENGTH + INDEX_LENGTH)/1024/1024/1024, 2) AS size_gb
FROM information_schema.TABLES WHERE TABLE_NAME = 'orders';
# SSD: ~1 GB/min | HDD: ~200 MB/min
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Use pt-online-schema-change (Percona):

```
$ pt-online-schema-change \
--alter "ADD COLUMN notes TEXT DEFAULT NULL" \
D=your_database,t=orders \
--host=localhost --user=root --ask-pass \
--chunk-size=1000 --max-lag=5 \
--critical-load="Threads_running=100" \
--progress=time,30 --execute
```

--max-lag pauses if replica lag exceeds threshold. Always test with --dry-run first.

Step 2: Use gh-ost (GitHub) — trigger-free:

```
$ gh-ost --host=localhost --user=root --password=pass \  
--database=your_database --table=orders \  
--alter="ADD COLUMN notes TEXT DEFAULT NULL" \  
--chunk-size=1000 --max-lag-millis=5000 \  
--serve-socket-file=/tmp/gh-ost.sock --execute
```

Step 3: Use native INSTANT DDL where possible (MySQL 8.0+):

```
mysql> ALTER TABLE orders ADD COLUMN flag TINYINT DEFAULT 0, ALGORITHM=INSTANT;  
  
# Adding index (INPLACE, no copy):  
mysql> ALTER TABLE orders ADD INDEX idx_status (status), ALGORITHM=INPLACE,  
LOCK=NONE;
```

③ Phase 3 — Confirmation

Step 1: Verify schema change applied:

```
mysql> SHOW CREATE TABLE your_database.orders\G  
mysql> CHECK TABLE your_database.orders EXTENDED;
```

Step 2: Verify replication caught up:

```
mysql> SHOW SLAVE STATUS\G  
# Seconds_Behind_Master: 0
```

CONFIRMED: Schema change applied, row counts match, CHECK TABLE OK, replication caught up.

Scenario 9: Binary Log Growth & Disk Space Exhaustion

Uncontrolled binary log growth silently fills the disk, causing MySQL crashes and refusal to restart.

① Phase 1 — How to Find the Issue

Step 1: Check disk space:

```
$ df -h /apps/mysql/binlog
$ du -sh /apps/mysql/binlog/
$ ls -lHS /apps/mysql/binlog/ | head -20
```

Step 2: Check binlog settings and log count:

```
mysql> SHOW VARIABLES LIKE 'log_bin%';
mysql> SHOW VARIABLES LIKE 'expire_logs_days';
mysql> SHOW VARIABLES LIKE 'binlog_expire_logs_seconds';
mysql> SHOW BINARY LOGS;
```

Step 3: Check if replicas still need old logs:

```
mysql> SHOW SLAVE HOSTS;
mysql> SHOW PROCESSLIST;
# Look for "Binlog Dump" threads – note Master_Log_File
# NEVER delete logs a replica still needs!
```

Step 4: Check error log for disk warnings:

```
$ grep -iE "disk full|no space|errno 28" /var/log/mysqld.log
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Emergency: purge old binary logs:

```
mysql> PURGE BINARY LOGS BEFORE NOW() - INTERVAL 3 DAY;

mysql> PURGE BINARY LOGS TO 'mysql-bin.000050';

# If MySQL cannot start (disk full):
$ cd /apps/mysql/binlog
$ rm mysql-bin.000001 mysql-bin.000002 mysql-bin.000003
$ vi /apps/mysql/binlog/mysql-bin.index # remove deleted entries
$ systemctl start mysql
```

WARNING: NEVER delete logs that replicas still need. Check SHOW SLAVE HOSTS first.

Step 2: Configure auto-expiration:

```
mysql> SET GLOBAL expire_logs_days = 7;

# MySQL 8.0+:
mysql> SET GLOBAL binlog_expire_logs_seconds = 604800;

# /etc/my.cnf: expire_logs_days = 7
# max_binlog_size = 256M
```

Step 3: Reduce binlog volume:

```
mysql> SET GLOBAL binlog_row_image = 'MINIMAL';
# Reduces size by 50-80%

# MySQL 8.0.20+ compression:
mysql> SET GLOBAL binlog_transaction_compression = ON;
```

Step 4: Set up disk monitoring:

```
$ cat > /usr/local/bin/check_binlog_space.sh << 'EOF'
#!/bin/bash
USAGE=$(df /apps/mysql/binlog --output=pcent | tail -1 | tr -d " %")
if [ "$USAGE" -gt 80 ]; then
echo "ALERT: Binlog at ${USAGE}%" | mail -s "MySQL Disk" dba@company.com
fi
EOF
$ chmod +x /usr/local/bin/check_binlog_space.sh
# Cron: */15 * * * * /usr/local/bin/check_binlog_space.sh
```

③ Phase 3 — Confirmation

Step 1: Verify disk usage healthy:

```
$ df -h /apps/mysql/binlog
# < 70% usage
mysql> SHOW BINARY LOGS;
# Oldest log within retention window
```

Step 2: Confirm auto-purge works:

```
mysql> FLUSH BINARY LOGS;
# Wait, then: SHOW BINARY LOGS;
# Old logs beyond retention should be gone
```


CONFIRMED: Disk < 70%, auto-purge active, oldest binlog within retention, alerts configured.

Scenario 10: Security Misconfig, Privilege Bloat & Audit Overhead

Over-permissive accounts, wildcard grants, missing hardening, and heavy audit logging degrade performance and security.

① Phase 1 — How to Find the Issue

Step 1: Audit user privileges for over-permissive grants:

```
mysql> SELECT User, Host, authentication_string != '' AS has_pw,  
account_locked FROM mysql.user ORDER BY User;  
  
mysql> SELECT * FROM information_schema.USER_PRIVILEGES  
WHERE PRIVILEGE_TYPE = 'SUPER' OR IS_GRANTABLE = 'YES';  
  
mysql> SHOW GRANTS FOR 'app_user'@'%';
```

Step 2: Check for no-password and anonymous accounts:

```
mysql> SELECT User, Host FROM mysql.user  
WHERE authentication_string = '' OR authentication_string IS NULL;  
  
mysql> SELECT User, Host FROM mysql.user WHERE User = '';  
# Anonymous users should NEVER exist in production
```

Step 3: Check audit plugin overhead:

```
mysql> SHOW PLUGINS;  
mysql> SHOW VARIABLES LIKE '%audit%';  
$ ls -lh /apps/mysql/data/audit.log  
$ du -sh /apps/mysql/data/server_audit*
```

② Phase 2 — Troubleshooting / Remediation

Step 1: Revoke over-permissive grants:

```
mysql> REVOKE ALL PRIVILEGES ON *.* FROM 'app_user'@'%';

mysql> GRANT SELECT, INSERT, UPDATE, DELETE ON your_database.* TO
'app_user'@'10.0.0.%';

# Read-only user for reporting:
mysql> CREATE USER 'report_user'@'10.0.0.%' IDENTIFIED BY 'StrongPass!';
mysql> GRANT SELECT ON your_database.* TO 'report_user'@'10.0.0.%';
mysql> FLUSH PRIVILEGES;
```

Step 2: Remove anonymous users, secure root:

```
mysql> DROP USER ''@'localhost';
mysql> DELETE FROM mysql.user WHERE User = 'root' AND Host != 'localhost';
mysql> SET GLOBAL default_password_lifetime = 90;
mysql> FLUSH PRIVILEGES;
```

Step 3: Optimize audit plugin:

```
mysql> SET GLOBAL server_audit_events = 'CONNECT,QUERY_DDL,QUERY_DCL';
# Avoids logging every SELECT/INSERT

mysql> SET GLOBAL server_audit_file_rotate_size = 104857600; -- 100MB
mysql> SET GLOBAL server_audit_file_rotations = 10;
```

Step 4: Enable SSL/TLS and harden:

```
mysql> ALTER USER 'app_user'@'%' REQUIRE SSL;
mysql> SET GLOBAL local_infile = OFF;
mysql> SET GLOBAL require_secure_transport = ON;

$ mysql_secure_installation
```

③ Phase 3 — Confirmation

Step 1: Verify no over-permissive grants:

```
mysql> SELECT GRANTEE, PRIVILEGE_TYPE FROM information_schema.USER_PRIVILEGES
WHERE PRIVILEGE_TYPE IN ('SUPER','ALL PRIVILEGES')
AND GRANTEE NOT LIKE "'root'%";
# Should be empty

mysql> SELECT User, Host FROM mysql.user WHERE User = '';
# Empty (no anonymous)
```

Step 2: Verify SSL enforced:

```
mysql> SHOW VARIABLES LIKE 'require_secure_transport';  
# Should be ON  
$ mysql -u app_user -p -h db-server --ssl-mode=REQUIRED -e 'SELECT 1;'
```

CONFIRMED: Least-privilege enforced, no anonymous users, audit overhead minimal, SSL active, server hardened.

Quick Reference: Key Parameters at a Glance

Parameter	Recommended	Impact
innodb_buffer_pool_size	70-80% of RAM	Read performance
innodb_log_file_size	1-2 GB	Write throughput
innodb_flush_log_at_trx_commit	1 (safe) / 2 (fast)	Commit speed vs safety
innodb_io_capacity	Match disk IOPS	Flush throughput
innodb_flush_method	O_DIRECT	No double buffering
max_connections	200-500	Connection capacity
wait_timeout	300	Idle connection cleanup
thread_cache_size	64-128	Thread reuse
tmp_table_size	256M	Reduce disk temp tables
sort_buffer_size	2-4M	Sort performance
innodb_lock_wait_timeout	10-30	Lock fail-fast
slave_parallel_workers	4-16	Replication speed
expire_logs_days	3-7	Binlog disk usage
binlog_row_image	MINIMAL	Reduce binlog size
long_query_time	1-2	Slow query capture
vm.swappiness (OS)	1	Prevent MySQL swap
innodb_print_all_deadlocks	ON	Deadlock visibility

Decision Matrix: Symptom → Scenario Mapping

Symptom	Likely Scenario
Single query slow / full table scan	Scenario 1 — Slow Query & Index
Buffer pool hit rate < 99%	Scenario 2 — Buffer Pool & Memory
Tables heavily fragmented / oversized .ibd	Scenario 3 — Table & Index Bloat
"Too many connections" / Threads spike	Scenario 4 — Connection Storms
Disk %util > 80%, high iowait	Scenario 5 — Disk I/O & Redo/Flush
Lock waits, deadlocks, metadata locks	Scenario 6 — Lock Contention
Seconds_Behind_Master > 0	Scenario 7 — Replication Lag
ALTER TABLE blocking for hours	Scenario 8 — Slow DDL
Disk full from binary logs	Scenario 9 — Binlog Disk Space

This runbook is a living document. Review and update after every workload change, hardware upgrade, or incident.